# High-Impedance Fault Region Locator as Firmware on a Simulated Microcontroller

Pedro Henrique Faluba Martins

Universidade Federal de Minas Gerais (UFMG), School of Engineering
Undergraduate Program in Electrical Engineering

Technical report

Belo Horizonte, 2026

**Abstract**

A previous work selected fifteen features, built on the discrete Fourier, Stockwell and wavelet transforms of substation measurements, for locating the region of a high-impedance fault, and argued that they are cheap to compute. This report tests that argument on an embedded target. The complete locator, feature extraction followed by a classifier, was implemented as C firmware for an ARM Cortex-M4F microcontroller (STM32F407) simulated in Renode, verified against a Python reference, and measured in flash, RAM, stack and executed instructions across 80 configurations of feature set, model, model complexity and compiler optimization. The cost lies in the feature extraction, 17.17 million instructions per window, 60% of it in the wavelet transform, while inference takes at most 0.02% of the total. The selected set needs 4.4 times fewer instructions than all 7056 candidate features without losing accuracy with a random forest (macro-F1 0.781), and outperforms every random set of the same size at comparable cost. Flash is set by the model and RAM by the extraction; storing the raw window is the main obstacle to a real device.

## 1 Introduction

High-impedance faults (HIFs) occur when an energized conductor touches a material of very low conductivity. They draw low, random and non-linear currents, which makes them hard to detect and locate with conventional protection (Baharozu et al., 2023). A previous work (Martins, 2026) proposed a systematic feature selection procedure for locating the region of an HIF from substation measurements alone. Its result was a set of fifteen feature families, extracted with the discrete Fourier, Stockwell and wavelet transforms over a 20-cycle window, which clearly outperformed random sets of the same size under nested cross-validation.

That work also argued that the selected features are cheap to compute, since extracting all 7056 features of one simulation took about 0.2 s on a general-purpose processor, less than the duration of the window itself. The argument, however, was made on a personal computer, whereas protection and monitoring equipment in substations is typically embedded, built on microcontrollers with memory in the hundreds of kilobytes and no operating system. Running the locator on the measuring device itself would remove the communication with a control centre and its latency, but it is not known whether the transforms used, in particular the Stockwell and wavelet transforms, fit the resources of such a device, nor whether feature selection saves resources compared with using all candidate features.

Moreover, the previous work delivered a feature set rather than a trained model, leaving as future work an intelligent algorithm fed by that set. The choice of algorithm also matters on the device, since different model families take memory and processing in very different proportions.

The objective of this work is therefore to implement the complete locator, that is, the extraction of the selected features and a downstream classifier, as C firmware for a simulated ARM Cortex-M4F microcontroller, and to measure the memory and computation it needs in different configurations. To this end, the embedded implementation is first verified against the Python reference implementation; then the selected set, all candidate features, random subsets and the top features of the ranking are compared, combined with four model families, different model complexities and two compiler optimization levels.

# 2 Background

This section covers machine learning on microcontrollers and its tools, the platform adopted and its libraries, the functional simulation used to run it, and how works on HIF location with intelligent algorithms treat computational cost.

## 2.1 Machine learning on microcontrollers

Running machine learning models on microcontrollers, known as TinyML, means inference on devices with memory in the kilobytes and power in the milliwatts, without an operating system and without a mandatory link to servers (Warden and Situnayake, 2019). According to Ray (2022), the field is motivated by lower latency, lower energy use and less dependence on connectivity, at the price of severe memory and processing constraints that call for dedicated models and implementations.

The main tools are:

- **TensorFlow Lite Micro:** an interpreter for neural networks trained in TensorFlow, aimed at microcontrollers and supporting 8-bit quantization, whose evaluation shows low resource use and small runtime overhead (David et al., 2021).
- **CMSIS-NN:** a library of neural network kernels optimized for Cortex-M processors, in 8- and 16-bit fixed point, reporting a 4.6-fold runtime gain over reference implementations (Lai et al., 2018).
- **emlearn:** a library that converts models trained in scikit-learn (Pedregosa et al., 2011), such as decision trees, random forests and neural networks, into C code without dynamic memory allocation (Nordby, 2019). Because it covers classical machine learning models and not only neural networks, it is the tool adopted here.

For evaluation, MLPerf Tiny defines a standard set of tasks and metrics, namely accuracy, latency and energy, measured on real devices (Banbury et al., 2021). Its tasks, however, are keyword spotting, visual wake words, image classification and anomaly detection, where the model's input is the signal itself or a simple spectrogram. Problems like the one studied here, where feature extraction involves time-frequency transforms over several channels, are not represented.

## 2.2 The Cortex-M4F platform and CMSIS-DSP

Cortex-M4 processors implement the 32-bit Armv7E-M architecture, with a three-stage pipeline, signal processing instructions and an optional single-precision floating-point unit (Arm, 2026a). The STM32F407 microcontroller pairs this processor, with the floating-point unit, with 1 MB of flash and 192 KB of RAM, of which 64 KB are core-coupled memory (CCM), accessible to the processor but not to the DMA controllers (STMicroelectronics, 2026). It is a widely used, low-cost microcontroller, which makes it representative of an embedded measuring device.

For signal processing, the CMSIS-DSP library provides routines optimized for the Cortex-M family, including the real fast Fourier transform (FFT) (Arm, 2026b). This transform, however, is only available for power-of-two lengths from 32 to 4096 points, a restriction that matters for the Stockwell transform, as discussed in Section 3.

### 2.3 Functional simulation of microcontrollers

Renode is a simulator that runs the binary compiled for the microcontroller unchanged, with models of each platform's peripherals, such as serial interfaces, and supports test automation through Robot Framework (Antmicro, 2026). It counts the instructions the processor executes, but it is functional rather than cycle-accurate: it reproduces the program's behaviour without modelling the pipeline, flash wait states or instruction latencies. Instruction counts are therefore suitable for comparing configurations, but they do not correspond to execution time on a real device.

### 2.4 Computational cost in HIF location

The literature on HIF location based on intelligent algorithms relies mostly on artificial neural networks (Farias et al., 2018; Ledesma et al., 2020; Sarangi et al., 2023), as well as support vector machines (Joga et al., 2023), Gaussian process regression (Moloi et al., 2018) and comparisons between algorithms (Nunes Lopes et al., 2025; Lopes, 2024), of which Lopes (2024) chooses an ensemble of decision trees. These works evaluate location performance, but treat the cost of running it qualitatively. Farias et al. (2018), for example, highlight the simplicity and low implementation cost of their method, which uses substation measurements only, without quantifying the memory or processing it requires.

In the comparison made by Martins (2026), the processing required by the region-oriented works is likewise described by the techniques used and the measurement points needed, and none of them reports memory or processing measurements on an embedded device. The previous work itself measured extraction time on a personal computer only.

Two observations frame the contribution of this work. First, TinyML tools and benchmarks focus on the model, whereas in HIF location the feature extraction involves time-frequency transforms potentially more expensive than the model itself. Second, HIF location works do not quantify the memory and processing needed on an embedded device. This work measures both, for the feature set obtained by Martins (2026).

## 3 Method

This section describes the data, feature sets and models evaluated, the platform and the C implementation of the extraction, and then the procedures used to verify that the implementations agree, the measurement method and the configuration sweep. Figure 1 summarizes the method.

### 3.1 Data and feature sets

The feature database built by Martins (2026) was used: 7056 features per simulation over 7990 HIF simulations on an adapted IEEE 34-node test feeder, at the 40 dB noise level adopted in that work. The train/test split was fixed once and kept for every experiment. It holds out seven of the 34 HIF signals for testing, drawn with a fixed seed, which yields 6345 training and 1645 test simulations with the same proportion of regions. Splitting by HIF signal rather than by simulation keeps the same arc waveform from appearing in both training and test sets.

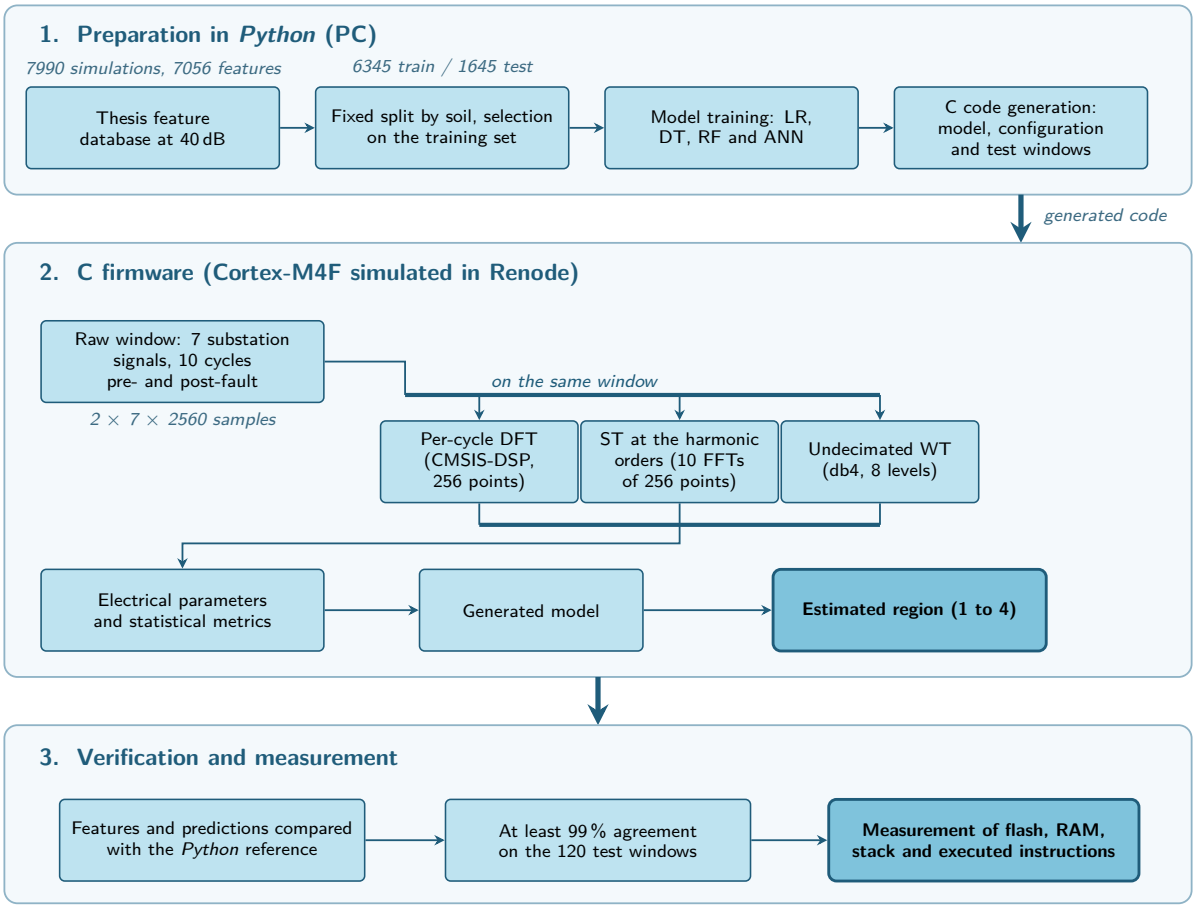



**Figure 1:** Implementation and evaluation flow of the locator as firmware.

The ranking published by Martins (2026) was obtained on all simulations, including those that form this work's test set. For that reason, that work's selection chain, namely correlation clustering, filtering, an embedded stage, a wrapper stage and stability selection, was re-run unchanged on the training simulations only, over one hundred runs. The resulting stability index was 0.256, below the 0.315 obtained on the full set, as expected given the fewer HIF signals in each subsample, and the resulting set shares ten of its fifteen clusters with the published one. This set, shown in Table 1, is the reference set of this work.

Besides the reference set, the following were evaluated: the set published by Martins (2026), kept only as a comparison because it was selected with access to the test simulations; all 7056 candidate features; the top three, five and ten features of the reference set; and ten subsets of fifteen features drawn uniformly from the 7056, each with a fixed seed.

## 3.2 Classification models

Four model families were evaluated, all trained with scikit-learn (Pedregosa et al., 2011) with class weighting:

- **Logistic regression (LR):** with L2 regularization, as the reference classifier of the wrapper stage in Martins (2026).
- **Decision tree (DT):** with a maximum depth of eight.
- **Random forest (RF):** thirty trees of maximum depth eight, predicting the mean of the leaves' class proportions.

**Table 1:** Fifteen-feature set selected on the training simulations.

| # | $\pi$ | Transform | Parameter | Channel | Order | Metric |
|---|---|---|---|---|---|---|
| 1 | 0.95 | DFT | Impedance magnitude | $V_f$ / $I_f$ | h1 | maximum |
| 2 | 0.76 | WT | Magnitude | $V_f$ | D3 | delta |
| 3 | 0.68 | ST | THD | $I_f$ | all | ratio |
| 4 | 0.48 | WT | Magnitude | $I_n$ | D7 | maximum |
| 5 | 0.43 | WT | Impedance magnitude | $V_f$ / $I_f$ | D6 | ratio |
| 6 | 0.38 | WT | Energy | $I_f$ | D8 | range |
| 7 | 0.35 | DFT | Impedance magnitude | $\Sigma V$ / $I_n$ | h3 | mean |
| 8 | 0.29 | ST | Impedance magnitude | $\Sigma V$ / $I_n$ | h3 | ratio |
| 9 | 0.25 | DFT | Energy | $V_f$ | h1 | maximum |
| 10 | 0.21 | ST | Energy | $V_f$ | h8 | mean |
| 11 | 0.21 | ST | Magnitude | $V_2$ | h12 | delta |
| 12 | 0.20 | ST | Angle | $I_n$ | h1 | mean |
| 13 | 0.19 | DFT | Angle | $V_f$ | h1 | mean |
| 14 | 0.19 | ST | Magnitude | $V_2$ | h10 | delta |
| 15 | 0.15 | DFT | Energy | $V_2$ | h14 | mean |

$\pi$: selection frequency over the hundred runs; DFT, ST and WT: discrete Fourier, Stockwell and wavelet transforms; THD: total harmonic distortion; $V_f$ and $I_f$: faulted-phase voltage and current; $I_n$: neutral current; $\Sigma V$: sum of the three phase voltages; $V_2$: negative-sequence voltage; h$n$: harmonic order; D$n$: dyadic band of the WT.

- **Artificial neural network (ANN):** a perceptron with one hidden layer of sixteen neurons and ReLU activation.

The LR and the ANN receive the features after the signed logarithmic transform and the standardization adopted by Martins (2026). The DT and the RF receive the raw features, since decision trees are invariant to monotonic transforms, which saves fifteen logarithms per window on the device.

The DT and the RF were converted to C with emlearn (Nordby, 2019). For the LR, which the library does not support, and for the ANN, the coefficients and a dedicated inference routine were generated. In the ANN's case, emlearn's routine requires its intermediate vectors to be as wide as the input layer, which, with all 7056 features, would take an extra 55 KB of RAM.

### 3.3 Platform and implementation of the extraction

The firmware targets the STM32F407, simulated in Renode from the STM32F4 Discovery board platform, and is compiled with size optimization, hardware single-precision floating point and removal of unused code. All variables are static, with no dynamic memory allocation. The simulator's platform model has more memory than the real microcontroller and lacks the CCM. The CCM was therefore added to the model, and the real limits of 1 MB of flash and 128 KB of main RAM were enforced at link time.

The C extraction is generic, that is, able to compute any of the 7056 features. The set to compute is described by a table generated automatically from the model's features, which lists, for each transform, the channels and the harmonic orders or bands needed. Only those combinations are computed, and the working buffers are sized by them. The three transforms were implemented as follows:

- **DFT:** applied cycle by cycle with the CMSIS-DSP real FFT, with 256 points.
- **ST:** in Martins (2026), each harmonic voice is obtained by a 2560-point inverse transform,

a ten-cycle length that is not a power of two. The 2560-point forward transform was obtained by decimation, from ten 256-point FFTs. Since only the mean of each voice over each cycle is used, the inverse transform was replaced by the closed form of Equation (1), where $H$ is the forward transform, $k_0 = 10h$ is the bin of harmonic $h$, $G_h$ is the transform's Gaussian window (Stockwell et al., 1996), $B$ is the mean of an exponential term over the 256 samples of a cycle and $c$ is the cycle. The sum was truncated at $|m| \leq 1.5\,k_0$, where the Gaussian window is $e^{-44}$, a negligible contribution in single precision.

$$X_{h,c} = \frac{2}{N} \sum_{|m| \leq 15h} H[k_0 + m]\, G_h[m]\, B[m]\, e^{j2\pi mc/10} \tag{1}$$

- **WT:** implemented in undecimated form, by circular convolution with the fourth-order Daubechies filters upsampled at each level, accumulating the RMS value of each cycle during the filtering itself, without storing the bands.

The negative-sequence components of the WT, which need a phase rotation through the Hilbert transform, use the 2560-point inverse transform obtained by the same decimation. All computations are in single precision.

### 3.4 Verification of equivalence

The reference implementation was rewritten in Python, independently of the code of Martins (2026), and reproduces that work's feature matrix, on all 1645 test simulations and all 7056 features, with a maximum relative error of $6.9 \times 10^{-8}$, the rounding of single-precision storage. Every C routine has a counterpart in this reference, and the reformulated algorithms, namely the decimation and the closed form of the ST, were checked against the definitions to within double-precision rounding.

The reference is computed on the same single-precision window the device receives, so only the arithmetic of the C implementation differs between the two. A feature computed in C, $c$, is considered equivalent to the reference, $p$, when $|c - p| \leq 10^{-3}|p| + 10^{-3}\sigma$, where $\sigma$ is the feature's standard deviation over the test set, with angles compared by wrapped difference. The term proportional to $\sigma$ is needed because delta, kurtosis and angle features take values close to zero, where relative error alone is unstable.

Each raw window takes 140 KB, so only six or seven windows fit in flash next to the program. A total of 120 windows from the test set, stratified by region, were run in batches, each compiled into a separate firmware image. A configuration was considered valid when the device's prediction agrees with scikit-learn's on at least 99% of these windows.

### 3.5 Measurement method

The metrics and how each was obtained:

- **Flash:** the sum of the code and constant data sections, taken from the linker map and split by module, namely extraction, CMSIS-DSP, model, standard library and application, excluding the test windows.
- **Static RAM:** the sum of initialized and uninitialized variables, including the CCM, obtained the same way.
- **Stack:** the peak usage, measured by filling the stack region with a known pattern beforehand and checking, at the end of the run, how much of it was overwritten.

- **Executed instructions:** read from the simulator's counter by a hook on the address of a marker function, which runs exactly when the processor reaches it. Extraction was measured as the mean over the batch's windows, each run twice, and inference as the mean over the reference features of up to 1645 test simulations, since a tree's cost depends on the path taken. One additional window was measured with markers between the extraction stages.

The counter reading was checked on a loop of known length. The alternative of pausing the simulation when a message appears on the serial interface proved imprecise, since the simulator only stops at its next synchronization point, up to millions of instructions later.

The number of instructions is not the execution time on a real device. On an STM32F407 at 168 MHz, time also depends on pipeline flushes at each branch, on flash wait states, only partly hidden by its accelerator, and on instruction latencies, from one cycle for simple integer operations to fourteen cycles for floating-point division and square root. The time estimates given here therefore assume one instruction per cycle and are only rough estimates.

### 3.6 Configuration sweep

The sweep evaluated 80 configurations in three groups, each varying one factor at a time:

- **Feature set and model:** the sixteen sets of Section 3.1, combined with the four model families, optimized for size, for 64 configurations.
- **Model complexity:** on the reference set, RF with 5, 10 and 30 trees and depths of 4, 8 and 12, DT with depths of 4 and 12 and ANN with 8 and 32 neurons.
- **Compiler optimization:** the four model families on the reference set, compiled with speed optimization.

Each configuration is trained, converted, run on the simulated device over the 120 windows and measured automatically. The window length was not varied, since the 256 samples per cycle and the 20-cycle window are part of the definition of the features in Martins (2026).

## 4 Results and discussion

This section presents the equivalence between the implementations, the cost of the reference configuration, and the effects of the feature set, the model complexity and the compiler optimization.

### 4.1 Equivalence between the implementations

The four model families, converted to C and fed with the reference set, agreed with scikit-learn on all 1645 test simulations, both compiled for the PC and run on the simulated device. Across the sweep, only eight of the 25 RF configurations disagreed, on one to four of the 1645 simulations, always identically on the PC and on the device. The disagreement comes from emlearn's 8-bit representation of the leaves' class proportions, which flips the decision when two regions are practically tied, and not from an error in the implementation.

The fifteen features of the reference set computed on the device met the equivalence criterion on all 1800 values of the 120 windows, the largest deviation being 11.5% of the tolerance. It occurs in the impedance magnitude between the voltage sum and the neutral current: the three phase voltages, of about 21 kV, cancel into a small residue whose single-precision resolution is limited.

Extended to all 7056 features, the check met the criterion on 99.99% of the values. The 244 features that failed on at least one window are concentrated in impedances between cancelling quantities, such as the voltage sum and the negative-sequence components, and in statistical moments of nearly constant series. In these features, the error of single-precision arithmetic is, in median, about seven times that caused by storing the window in single precision alone, which is a limit of the numerical representation rather than of the implementation. Even so, all 80 configurations of the sweep met the prediction agreement criterion, with a minimum of 99.17%, one disagreement in 120 windows, in one of the RFs mentioned above.

## 4.2 Cost of the reference configuration

Table 2 shows the resources needed by the reference set with each model family.

**Table 2:** Resources needed by the reference set.

| | LR | DT | RF | ANN |
|---|---|---|---|---|
| macro-F1 | 0.680 | 0.741 | 0.781 | 0.689 |
| Total flash (KB) | 18.8 | 21.9 | 131.5 | 19.9 |
| Model flash (KB) | 0.6 | 3.7 | 113.2 | 1.7 |
| Static RAM (KB) | 97.3 | 97.3 | 97.3 | 97.5 |
| Peak stack (B) | 1800 | 1800 | 1800 | 1800 |
| Instructions per extraction (millions) | 17.17 | 17.17 | 17.17 | 17.17 |
| Instructions per inference | 1648 | 219 | 2717 | 3828 |

macro-F1 on the 1645 test simulations.

Feature extraction takes 17.17 million instructions per window, while inference takes 219 to 3828 instructions, about 0.02% of the total in the worst case. The locator's cost is therefore set by the extraction, not by the model. Even in the LR, the smallest model in memory, most of the inference instructions go to the fifteen logarithms of the input transform rather than to the linear combination itself. Figure 2 breaks this cost down by extraction stage.

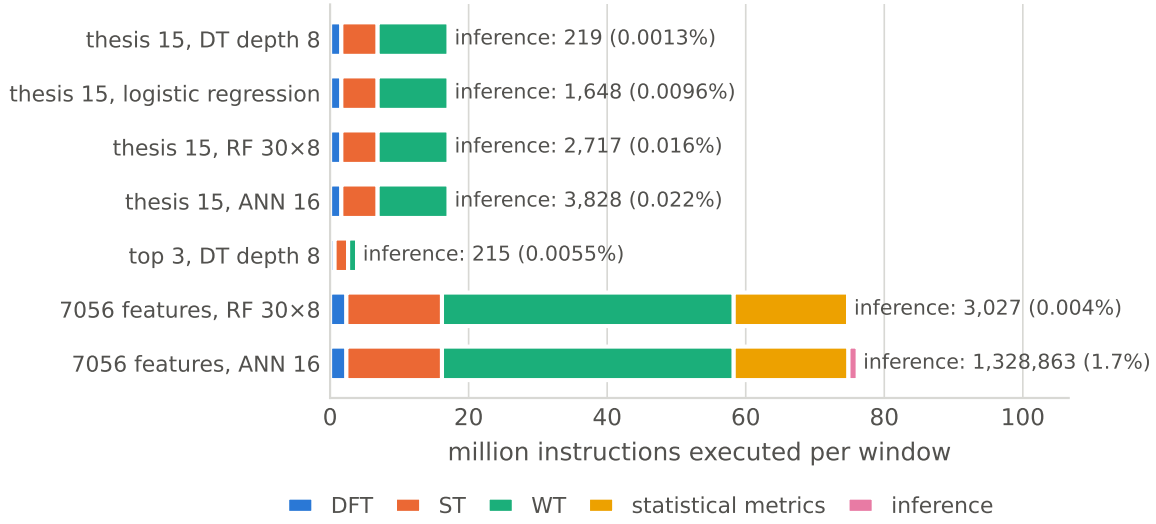



**Figure 2:** Instructions executed per window, by extraction stage and in inference.

For the reference set, the WT accounts for 60% of the extraction instructions, the ST for 31%, the DFT for 9% and the statistical metrics for only 0.2%. The weight of the WT comes from its undecimated form, in which each of up to eight levels filters all 2560 samples of each window. The ST, whose definition requires one inverse transform per harmonic, accounts for less than a

third of the total thanks to the closed form of Equation (1), even though the THD is computed over all twenty harmonic orders. The implementation itself affects these numbers: removing the circular index handling from the interior of the WT filtering cut that stage's instructions by 32%, with identical results.

As for memory, the flash of the reference set is dominated by the model in the RF's case, 113.2 KB, while the extraction takes 4.0 KB, CMSIS-DSP 5.8 KB and the standard library 7.3 KB. Static RAM, about 97 KB for every family, is almost entirely the extraction's working buffers, namely each channel's transform coefficients, the ST's Gaussian tables and the WT's intermediate vectors. As a rough estimate, at one instruction per cycle at 168 MHz, the extraction would take about 102 ms, below the roughly 333 ms of signal contained in the two windows.

### 4.3 Effect of the feature set

Table 3 shows the performance of each feature set with each model family, with the extraction instructions, and Figure 3 relates performance to computational cost for all configurations.

**Table 3:** Performance and extraction cost by feature set.

| Set | LR | DT | RF | ANN | Instructions (millions) |
|---|---|---|---|---|---|
| Reference (15) | 0.680 | 0.741 | 0.781 | 0.689 | 17.2 |
| Published (15) | 0.694 | 0.706 | 0.755 | 0.737 | 16.4 |
| Top 10 | 0.639 | 0.699 | 0.776 | 0.669 | 16.0 |
| Top 5 | 0.577 | 0.687 | 0.706 | 0.594 | 11.8 |
| Top 3 | 0.467 | 0.572 | 0.577 | 0.488 | 3.9 |
| Random (15), mean | 0.389 | 0.423 | 0.452 | 0.402 | 18.9 |
| Random (15), maximum | 0.429 | 0.484 | 0.491 | 0.445 | 33.8 |
| All (7056) | 0.653 | 0.780 | 0.754 | 0.632 | 74.8 |

macro-F1 on the 1645 test simulations. Extraction instructions per window; for the random sets, median and maximum over the ten draws.

The reference set outperformed all ten random sets with every model family: even its worst result, 0.680 with the LR, beats the best random result, 0.491 with the RF. This is all the more relevant since the random sets take between 7.3 and 33.8 million instructions, with a median of 18.9 million, a cost comparable to the reference set's. Selection, therefore, does not reduce cost relative to an arbitrary set of the same size, but concentrates in it the information about the region.

Compared with the 7056 candidate features, the reference set needs 4.4 times fewer instructions and loses no performance. The RF reaches 0.781 with the reference set and 0.754 with all features, while the DT reaches 0.741 and 0.780, respectively, the only family where the full set is better. In addition, the full set needs 139 to 167 KB of static RAM, which fits the microcontroller only by using the CCM, and 63 to 557 KB of flash, depending on the model. When extracting all 7056 features, the statistical metrics account for 22% of the instructions, against 0.2% for the reference set.

The top features of the ranking show that cost can be reduced with a gradual loss of performance. The top ten reach 0.776 with the RF, practically the same as the full set of fifteen, with 16.0 million instructions, while the top five reach 0.706 with 11.8 million and the top three 0.577 with 3.9 million. The drop is sharpest for the top three because they skip most of the WT bands and ST voices. The published set performed close to the reference set, better with the LR and the ANN and worse with the DT and the RF, and this comparison is optimistic for the published set, which was selected with access to the test simulations.

Figure 4 relates the same performance to flash. Unlike instructions, flash varies little with the

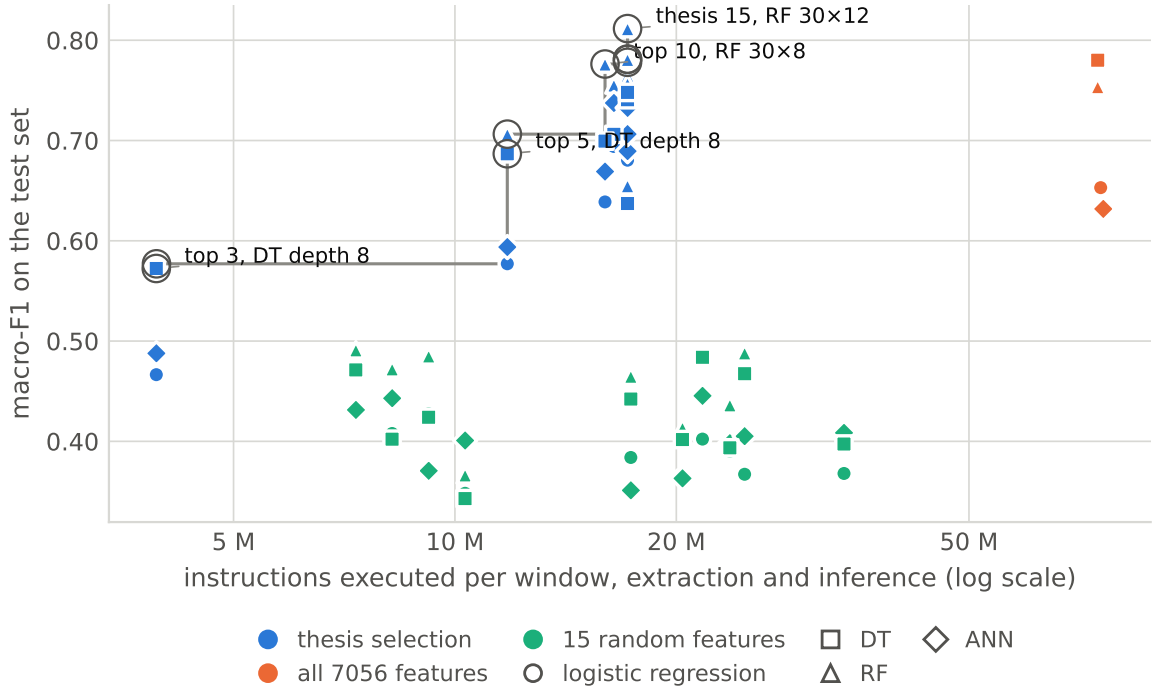



**Figure 3:** Performance against instructions executed per window. Circled points form the front of configurations not beaten in both performance and cost at once.

feature set, staying between 18 and 22 KB for every fifteen-feature set except with the RF. This is because the extraction code is the same for any set; only the table that configures it changes. Flash is thus set by the model, while feature selection sets the instructions executed and the RAM.

### 4.4 Model complexity and compiler optimization

For the RF, going from 5 to 30 trees and from depth 4 to 12 raised macro-F1 from 0.655 to 0.812, the best result of the sweep, at the cost of 314 KB of flash for the model, against 2.4 KB for the smallest configuration. The gains, however, are not uniform. With ten trees of depth eight, the RF reaches 0.778 with 38 KB, while thirty trees of the same depth add only 0.003 for 75 KB more. For the DT, depth 4 gave 0.637 and depth 12 gave 0.748, close to the 0.741 of depth 8. For the ANN, eight, sixteen and thirty-two neurons gave 0.706, 0.689 and 0.733, with no monotonic trend. In every case, inference stayed below 6300 instructions, so model complexity essentially affects flash.

Speed optimization raised the extraction instructions by 3.2%, from 17.17 to 17.71 million, cut the DT's inference instructions by 31% and added 1.3 to 9.3 KB of flash. More extraction instructions do not mean slower code, since this optimization level rearranges the code to save cycles in branches and memory accesses, an effect the functional simulator does not capture. This result illustrates the limitation discussed in Section 3.5.

### 4.5 Fit to embedded devices

The firmware of all 80 configurations fit the STM32F407, with at most 557 KB of flash and 169 KB of static RAM plus stack, the CCM being needed only for the 7056 features. No configuration would fit a microcontroller with 32 KB of RAM, typical of the Cortex-M0+ class, since the smallest static RAM observed was 49 KB.

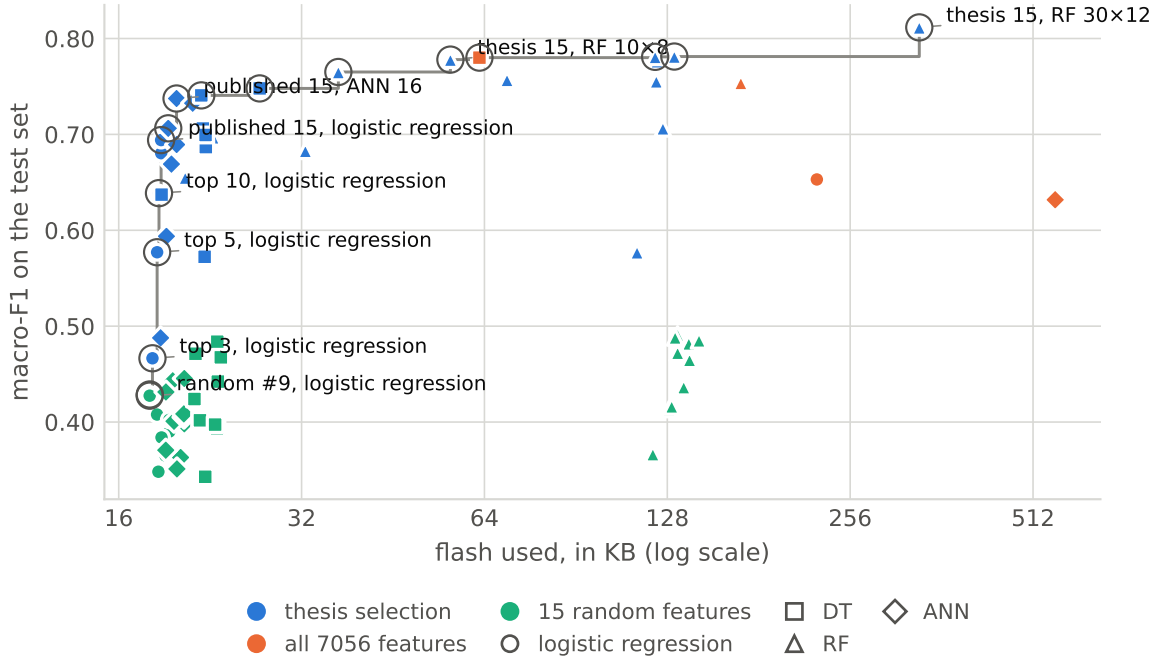



**Figure 4:** Performance against flash used.

One requirement, however, is not counted in these figures. In this work the raw windows live in flash as test data, but a real device would have to keep them in RAM as they are acquired. The two windows of the seven signals take 120 to 140 KB in single precision, depending on the channels the set needs, which, added to the extraction's working buffers, exceeds the RAM of the microcontroller adopted.

## 5 Conclusion

This work implemented the HIF region locator as C firmware for a simulated ARM Cortex-M4F microcontroller, from the feature set obtained by Martins (2026), and measured the memory and computation it needs in 80 configurations. To keep the test set unseen, that work's selection chain was re-run on the training simulations only, and the resulting set was combined with logistic regression, a decision tree, a random forest and an artificial neural network.

The extraction was built on the CMSIS-DSP library, whose fast Fourier transform is limited to power-of-two lengths. The Stockwell transform, defined over a 2560-sample window, was obtained by decimation and by a closed form for the mean of each voice over each cycle, which avoids the inverse transforms, and the undecimated wavelet transform accumulates each cycle's RMS value during the filtering. In every configuration, the device's predictions agreed with the reference implementation's on at least 99% of the test windows, and the features of the reference set were reproduced within tolerance on every window.

The results show that the locator's cost lies in the feature extraction, not in the model. For the reference set, extraction takes 17.17 million instructions per window, 60% of them in the wavelet transform, while inference takes at most 0.02% of the total. Feature selection cut this cost 4.4-fold compared with the 7056 candidate features with no loss of performance for the random forest, which reached a macro-F1 of 0.781, and outperformed every random set of the same size, whose cost is comparable. For the random forest, the top ten features of the ranking kept practically the same performance at a slightly lower cost. The choice of model, in turn, sets the flash, from 0.6 KB for logistic regression to 314 KB for the largest forest evaluated, which gave the best result, 0.812. The low computational cost argued in Martins (2026) thus holds on

a microcontroller, with the caveat that the extraction, not the model, should be the focus of any optimization.

Some limitations must be considered. The simulator is functional, so executed instructions allow configurations to be compared but do not determine execution time, which depends on memory wait states and instruction latencies on the real device. In addition, the raw windows live in flash as test data, whereas a real device would have to store them in RAM, which exceeds the capacity of the microcontroller adopted. Finally, the signals come from simulations, the faulted phase is taken from metadata, as in Martins (2026), and energy consumption was not measured.

### Future work

The results open several paths for continuing this work:

- **Running on a real device:** running the same firmware on an STM32F4 board, reading the processor's cycle counter and measuring energy consumption, would turn the instructions measured here into time and energy, and check the real-time requirement.
- **Continuous signal acquisition:** replacing the stored windows by continuous acquisition, with analog-to-digital converters and direct memory transfer, and processing cycles as they arrive, would reduce the memory needed for the raw window, the main obstacle identified.
- **Reducing the extraction cost:** since extraction dominates the cost, optimizing the wavelet transform, shrinking the working buffers to the orders actually used and using fixed-point arithmetic could reduce instructions and RAM, as long as equivalence with the reference is preserved.
- **Comparison with quantized neural networks:** comparing with models run by TensorFlow Lite Micro (David et al., 2021) in 8 bits would show the trade-off between precision and memory, even though inference is a small share of the total cost.
- **Including cost in feature selection:** the extraction cost of each feature, measured here, could become a criterion of the selection itself, favouring, among features of equivalent performance, those that are cheaper on the device.

## References


Antmicro. Renode documentation, 2026. URL `https://renode.readthedocs.io`.

Arm. Cortex-M4 technical reference manual, 2026a. URL `https://developer.arm.com/documentation/100166/0001`.

Arm. CMSIS-DSP: Embedded compute library for Cortex-M and Cortex-A, 2026b. URL `https://github.com/ARM-software/CMSIS-DSP`.

Eren Baharozu, Suat Ilhan, and Gurkan Soykan. High impedance fault localization: A comprehensive review. *Electric Power Systems Research*, 214:108892, 2023. ISSN 0378-7796. doi: https://doi.org/10.1016/j.epsr.2022.108892. URL `https://www.sciencedirect.com/science/article/pii/S0378779622009439`.

Colby Banbury, Vijay Janapa Reddi, Peter Torelli, et al. MLPerf tiny benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. URL `https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html`.

Robert David, Jared Duke, Advait Jain, et al. TensorFlow Lite Micro: Embedded machine learning for TinyML systems. In *Proceedings of Machine Learning and Systems*, volume 3, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/6c44dc73014d66ba49b28d483a8f8b0d-Abstract.html.

Patrick E. Farias, Adriano Peres de Morais, Jean Pereira Rossini, and Ghendy Cardoso. Non-linear high impedance fault distance estimation in power distribution systems: A continually online-trained neural network approach. *Electric Power Systems Research*, 157:20–28, 2018. ISSN 0378-7796. doi: https://doi.org/10.1016/j.epsr.2017.11.018. URL https://www.sciencedirect.com/science/article/pii/S0378779617304698.

S. Ramana Kumar Joga, Pampa Sinha, and Manoj Kumar Maharana. A novel graph search and machine learning method to detect and locate high impedance fault zone in distribution system. *Engineering Reports*, 5(1):e12556, 2023. doi: https://doi.org/10.1002/eng2.12556. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/eng2.12556.

Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs. arXiv:1801.06601, 2018. URL https://arxiv.org/abs/1801.06601.

Jorge Javier Gimenez Ledesma, Kelvin Bryan do Nascimento, Leandro Ramos de Araujo, and Débora Rosana Ribeiro Penido. A two-level ANN-based method using synchronized measurements to locate high-impedance fault in distribution systems. *Electric Power Systems Research*, 188:106576, 2020. ISSN 0378-7796. doi: https://doi.org/10.1016/j.epsr.2020.106576. URL https://www.sciencedirect.com/science/article/pii/S0378779620303801.

Gabriela Nunes Lopes. *Detecção e Localização de Faltas de Alta Impedância em Sistemas de Distribuição de Energia Elétrica com Geração Distribuída*. PhD thesis, Universidade de São Paulo, Escola de Engenharia de São Carlos, 2024.

Pedro Henrique Faluba Martins. Seleção de Atributos para Estimação de Regiões com Faltas de Alta Impedância [Feature Selection for Region Locating of High Impedance Faults]. Undergraduate thesis (electrical engineering), Universidade Federal de Minas Gerais, School of Engineering, Belo Horizonte, 2026.

K. Moloi, J. A. Jordaan, and Y. Hamam. High impedance fault classification and localization method for power distribution network. In *2018 IEEE PES/IAS PowerAfrica*, pages 84–89, 2018. doi: 10.1109/PowerAfrica.2018.8520972.

Jon Nordby. emlearn: Machine learning inference engine for microcontrollers and embedded devices. Zenodo, 2019. URL https://github.com/emlearn/emlearn.

Gabriela Nunes Lopes, Pedro I. N. Barbalho, José Carlos Melo Vieira, and Denis V. Coury. High impedance fault location in distribution systems: A novel approach with enhanced metrics and intelligent algorithms. *IEEE Access*, 13:108466–108480, 2025. doi: 10.1109/ACCESS.2025.3582359.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Partha Pratim Ray. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University – Computer and Information Sciences*, 34(4):1595–1623, 2022. doi: 10.1016/j.jksuci.2021.11.019.

Swetalina Sarangi, Binod Kumar Sahu, and Pravat Kumar Rout. High-impedance fault identification and location by using mode decomposition integrated adaptive multi-kernel extreme learning machine technique for distributed generator-based microgrid. In *Electrical Engineering*, volume 105, page 406, 2023. doi: https://doi.org/10.1007/s00202-022-01658-6.

STMicroelectronics. RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 advanced Arm-based 32-bit MCUs, 2026. URL `https://www.st.com/resource/en/reference_manual/rm0090-stm32f405415-stm32f407417-stm32f427437-and-stm32f429439-advanced-armbased-32bit-mcus-stmicroelectronics.pdf`.

R. G. Stockwell, L. Mansinha, and R. P. Lowe. Localization of the complex spectrum: the S transform. *IEEE Transactions on Signal Processing*, 44(4):998–1001, 1996. doi: 10.1109/78.492555.

Pete Warden and Daniel Situnayake. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, Sebastopol, 2019.